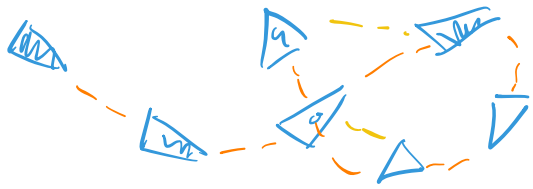
Graph Based SLAM using Pose Graphs

Poses



Constraints

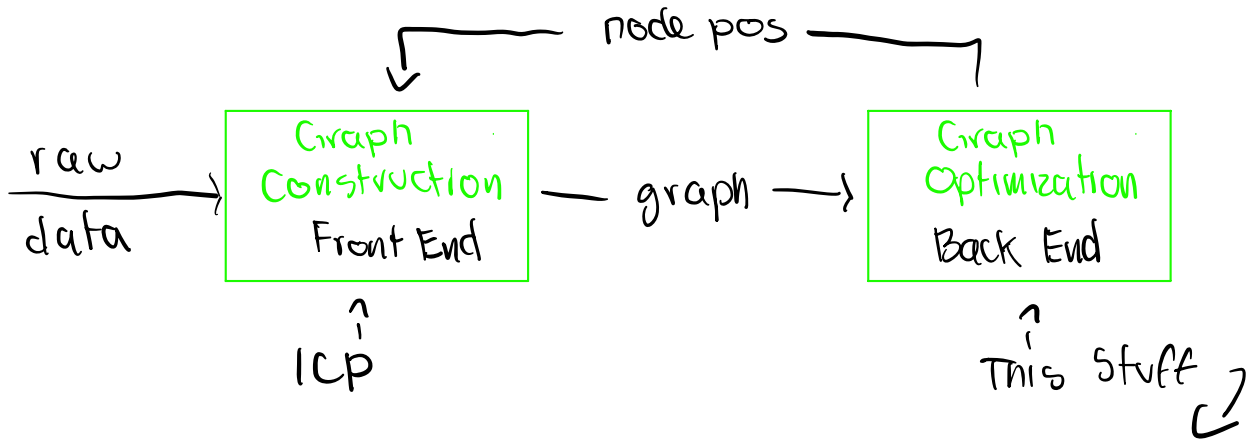
- Uncertain
- Loop Closing constraints b.w non sequential poses.
  - ICP

Graph

- Node: Pose
- Edge: Constraint

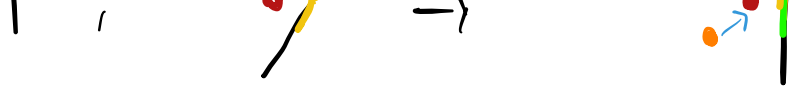
GOAL: Build graph, find node config to minimize error introduced by constraints

Overall System



Edge x_i, x_j

- exists if robot moves from $x_i \rightarrow x_{i+1}$
  - odometry measurement
- exists if robot observes same part of env from pose x_i and x_j



this is a **virtual measurement**

Transformations homogeneous coords

- Odometry Based Edge $(x_i^{-1} x_{i+1})$: how node i sees node $i+1$

Observation Based Edge $(x_i^{-1} x_j)$: how node i sees node j

Homo Coords

- N Dim expressed in N+1 Dim
  - → 3D expressed in 4D

} Extra dim allows to do trans + rot in single matrix

To HC: $(x, y, z)^T \rightarrow (x, y, z, 1)^T$ From HC $(x, y, z, w)^T \rightarrow \left(\frac{x}{w}, \frac{y}{w}, \frac{z}{w}\right)^T$

$$T = \begin{bmatrix} R_{3\times3} & t_{3\times1} \\ 0_{1\times3} & 1 \end{bmatrix}$$

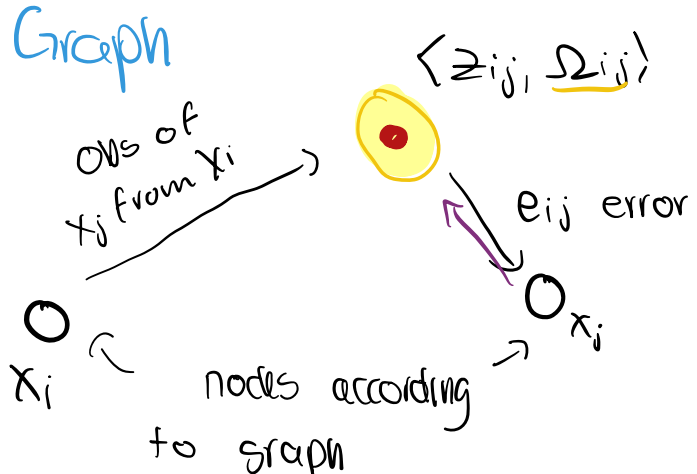
- $T^{\text{world}}_{\text{sensor}} = T^{\text{world}}_{\text{robot}} \cdot T^{\text{robot}}_{\text{sensor}}$

Information Matrix $\Omega_{i,j}$

how much we trust observation

- Due to noise in obv
- Bigger Ω = matters more

Pose Graph



We will want to move x_j like this

Least Squares $x^* = \underset{x}{\operatorname{argmin}} \sum_k e_k^T(x) \Omega_k e_k(x)$

- State Vector : $x^T = (x_1^T \; x_2^T \; \dots \; x_n^T)$ One vector for each node of graph
- Error Func

$$e_{ij}(x) = \underbrace{t2v}\left(\underbrace{z_{ij}^{-1}}\;\underbrace{(x_i^{-1} x_j)}\right)$$

how x_i sees x_j wrt graph

how x_i sees x_j wrt measurement, from ICP

1. Forwar trans what graph tels
2. Backward trans what obv tells me
3. what is diff = error

transformation to vector : $\begin{bmatrix} 3\times3 \text{ homo} \end{bmatrix} \to \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}$

- $e = 0$ if $z_{ij} = x_i^{-1} x_j$

Now, we get into Gauss Newton

Linearizing Error Func

$$e_{ij}(x + \Delta x) \approx e_{ij}(x) + \underbrace{J_{ij}}\Delta x \qquad J_{ij} = \frac{\partial e_{ij}(x)}{\partial x}$$

but e_{ij} only depends on few state vars, so most of $J = 0$

$\to J_{ij} = (0 \dots A_{ij} \dots B_{ij} \dots 0)$ Jacobian Sparsity

Why Important

Recall $b_{ij}^T = \sum e_{ij}^T \Omega_{ij} J_{ij}$

$H_{ij} = \sum J_{ij}^T \Omega_{ij} J_{ij}$ will lead to sparse H

- Only $\neq 0$ at ii, jj, ij, ji

We can use sparse solvers which are more efficient

Building Linear System

For Each Constraint

① Compute error $e_{ij} = t2v(z_{ij}^{-1}(x_i^{-1} x_j))$

② Compute Blocks of Jacobian

$$A_{ij} = \frac{\partial e(x_i, x_j)}{\partial x_i} \qquad B_{ij} = \frac{\partial e(x_i, x_j)}{\partial x_j}$$

③ Update Coeff Vector $\quad b_i^T = e_{ij}^T \Omega_{ij} A_{ij} \qquad b_j^T = e_{ij}^T \Omega_{ij} B_{ij}$

④ Update Normal Eq Matrix

- $H^{ii} += A_{ij}^T \Omega_{ij} A_{ij}$ $\qquad H^{ij} += A_{ij}^T \Omega_{ij} B_{ij}$
- $H^{ji} += B_{ij}^T \Omega_{ij} A_{ij}$ $\qquad H^{jj} += B_{ij}^T \Omega_{ij} B_{ij}$

Algo

While not Converged

$(H, b) =$ Build Linear Sys

$\Delta x =$ solve Linear Sys

$x = x + \Delta x$

end

return x

1D Example

2 nodes 1 obv

- $x = (x_1 \ x_2)^T$

$$x_1 \xrightarrow{z_{12} = 1} x_2$$

z tells us x_2 is 1m from x_1.
In reality, it is 0m

- $z_{12} = 1$ $\quad$ - $\Omega = 2$

- $e_{12} = z_{12} - (x_2 - x_1) = 1 - 0 = 1$

- $J_{12} = \left(\frac{\partial e}{\partial x_1}, \frac{\partial e}{\partial x_2} \right) = \begin{pmatrix} 1 & -1 \end{pmatrix}$

$\rightarrow b_{12}^T = e_{12}^T \Omega_{12} J_{12} = \begin{pmatrix} 2 & -2 \end{pmatrix}$ $\quad$ $\rightarrow H_{12} = J_{12}^T \Omega_{12} J_{12} = \begin{pmatrix} 2 & -2 \\ -2 & 2 \end{pmatrix}$

Finally $\Delta x = -H_{12}^{-1} b_{12}$ $\quad$ BUT $\det(H) = 0$ = can't invert

We need to fix one of the nodes

$H = \begin{pmatrix} 2 & -2 \\ -2 & 2 \end{pmatrix} + \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$ then $\Delta x = (0 \; 1)^T$ $\quad$ Update x_1 by 0, x_2 by 1

A Priori

We notice we need to have some known vars to fix everything. We want to update everything BUT these. How?

$\rightarrow$ We can just suppress those rows n cols